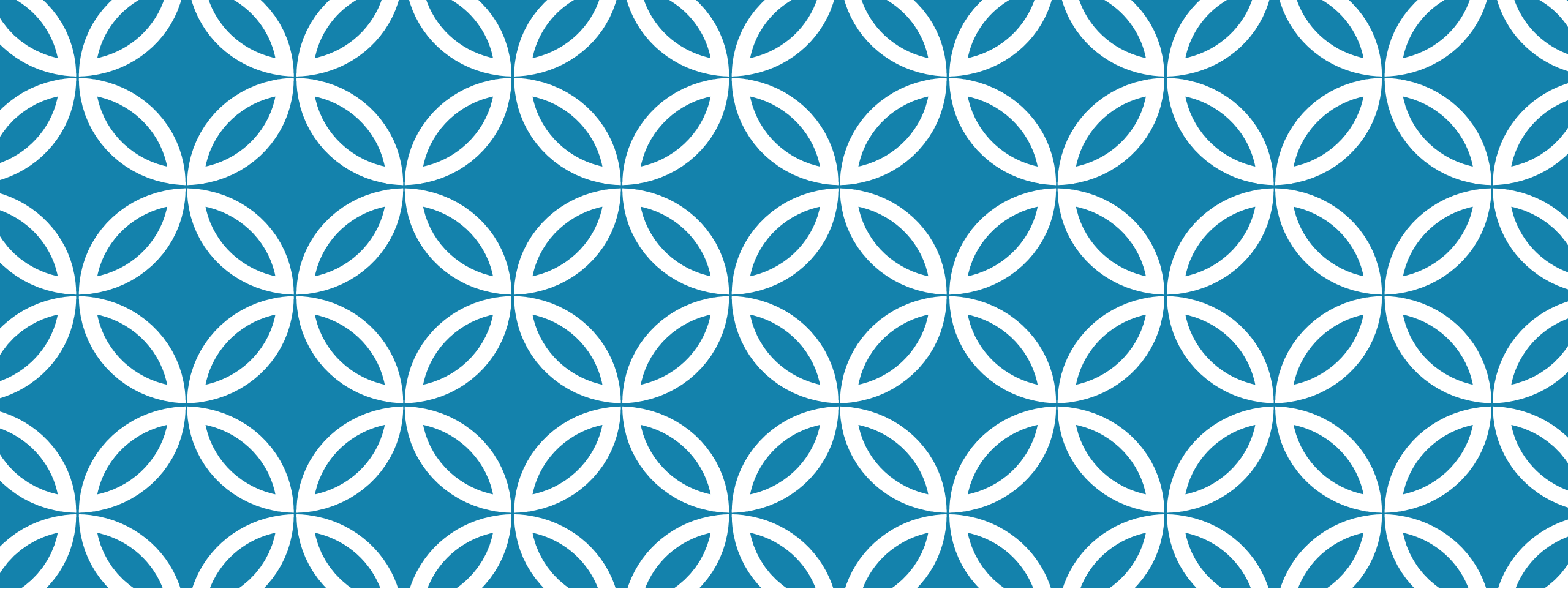




CSE455 - SOFTWARE TESTING

Chapter 2

QUALITY REQUIREMENTS

Products are usually spot-checked to make sure they fulfill the planned requirements in industry

If the final product is flawed or faulty, the production process or the design has to be modified to remedy this

SOFTWARE IS INTANGIBLE

Product itself isn't actually tangible, making “hands-on” testing impossible

Visual checks are limited and can only be performed by careful scrutiny of the development documentation

FAULTY SOFTWARE IS A SERIOUS PROBLEM

Bad software costs time and money and can ruin a company's reputation

Can endanger human life—for example, when the “autopilot” software in a partially autonomous vehicle reacts erroneously or too late

TESTING HELPS TO ASSESS SOFTWARE QUALITY

Check the quality of a software product to minimize the risk of failures or crashes

It reduces risk by revealing faults at the development stage

RISK OF USING FAULTY SOFTWARE

Every release of the VSR-II system has to be suitably tested before it is delivered and rolled out. This aims to identify and remedy faults before they can do any damage. For example, if the system executes an order in a faulty way, this can cause serious financial problems for the customer, the dealer and the manufacturer, as well as damaging the manufacturer's image. Undiscovered faults like this increase the risk involved in running the software.

TESTING INVOLVES A SPOT-CHECK APPROACH

Testing is often understood as spot-check execution of the software in question (the test object) on a computer.

The test object is fed with test data covering various test cases and is then executed

TESTING INVOLVES MORE THAN JUST EXECUTING TESTS

Involves a range of separate activities:

- Performing test
- Checking results
- Test planning
- Test analysis
- Design & Implementation of test cases
- Writing reports on test progress
- Risk analysis

NO LARGE SYSTEM IS FAULT-FREE

Software systems go live, even though faults still occur for certain combinations of input data

You cannot be sure that additional tests won't reveal previously undiscovered faults

DEFECT

Failure to fulfill a predefined requirement, or a discrepancy between the actual behavior (at run time or while testing) and the expected behavior (as defined in the specifications, the requirements, or the user stories)

When does the system's behavior fail to conform to its actual requirements?

FAULT CAUSES FAILURE

Failure is caused by a fault in the software, and the resulting condition is considered to be a defect.

The word “bug” is also used to describe defects that result from coding errors, such as an incorrectly programmed or forgotten instruction in the code

A fault is always the result of an error or a mistake made by a person—for example, due to a programming error at the development stage

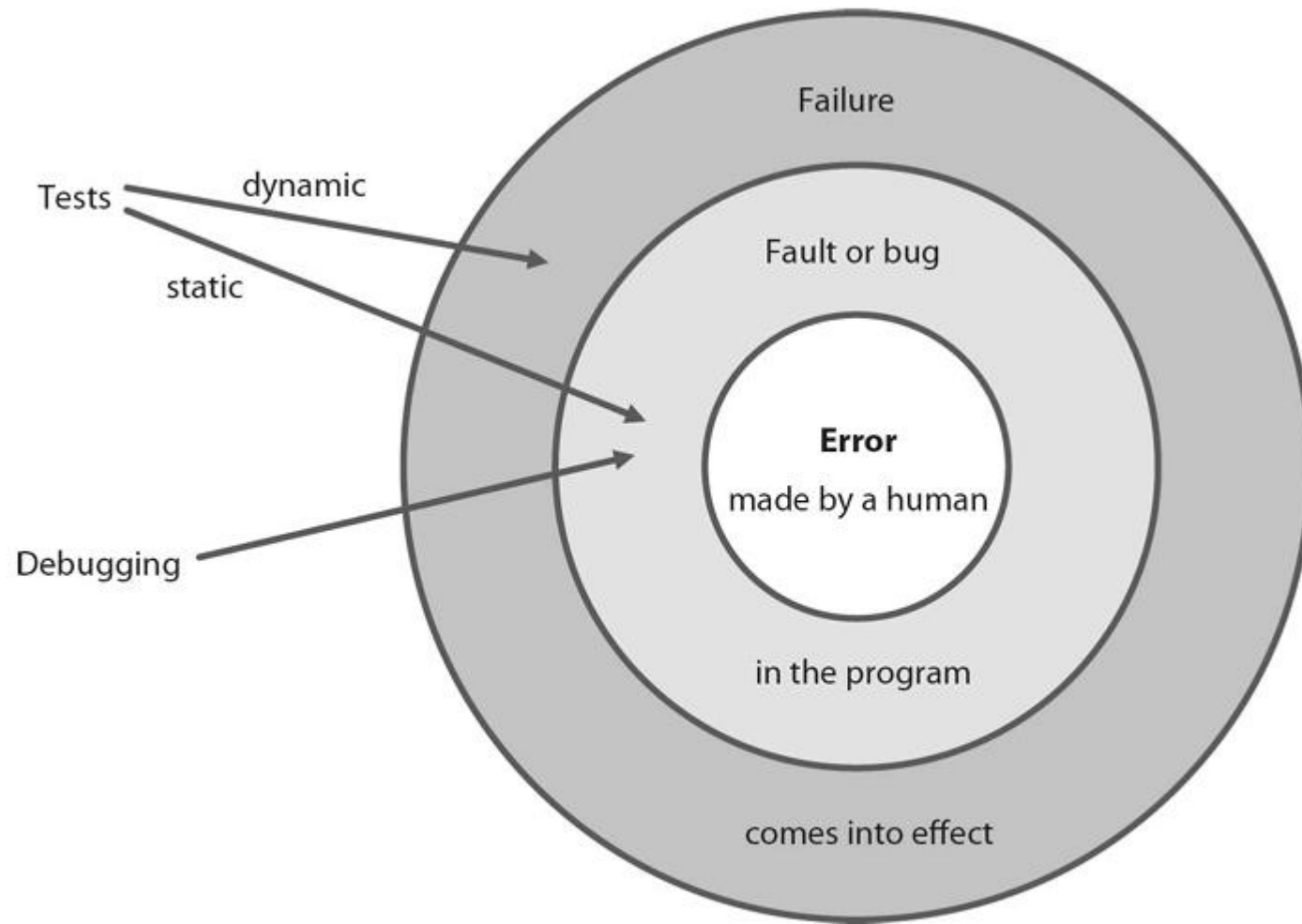


Fig. 2-1 *The relationships between, errors, faults, and failures*

PEOPLE MAKES ERRORS

Some typical causes of error are:

- Time pressure
- Complexity
- Misunderstanding b/w participants, different interpretation
- Technologies previously unknown to project participants
- Lack of participants training, inexperience team members

FALSE POSITIVE VS FALSE NEGATIVE

A test will indicate a failure even though the underlying fault (or its cause) isn't part of the test object. Such a result is known as a “false positive”

A failure doesn't occur even though testing should reveal its presence. This type of result is known as a “false negative”

Aspect	Verification	Validation
What it checks?	Are we building product correctly	Are we building the right product?
Focus	Process-oriented (checks documentation, design, and code before execution)	Product-oriented (checks actual product after execution)
When	Before implementation (Static Testing)	After implementation (Dynamic Testing)
How	Reviews, inspections, walkthroughs	Functional testing, system testing, UAT (User Acceptance Testing)
Example	Checking a software design document to ensure all requirements are covered before coding	Testing the developed software to see if it meets customer needs and works as expected

CASE STUDY: VAGUE REQUIREMENTS AS A CAUSE OF SOFTWARE FAULT

Interest rate the system uses is stored in a table, although the purchase of vehicles involved in promotions and special offers can be subject to differing interest rates. VSR-II is to include the following additional requirement:

REQ: *If the customer agrees to and passes an online credit check, the EasyFinance module applies an interest rate from a special bonus interest rate table.*

The author of this requirement unfortunately forgot to clarify that a reduction in the interest rate is not permissible for vehicles sold as part of a special offer. This resulted in this special case not being tested in the first release. In turn, this meant that customers were offered low interest rates online but were charged higher rates when billed, resulting in complaints.

TESTING IS NOT DEBUGGING

Debugging is usually performed by developer while testing by a professional tester, while these roles can switch in a certain development methodology

Debugging is about find the exact line # of code (location) of error

Finding and correcting faults is debugging

Debugging pinpoints software faults, testing is used to reveal effect of fault causes

CONFIRMATION TESTING

Tests used to check that a fault has been successfully remedied

Fault correction often leads to the creation of new faults that are only revealed when completely new input scenarios are used

Need to write new tests that check for unwanted side effects of the correction process

OBJECTIVES OF TESTING

Qualitative evaluation of work product

Prove that all specific requirements have been completely implemented and that the test object functions as expected for the users and other stakeholders

Level of quality-related risk can be reduced through identification and correction of software failures

Analysis of the program and its documentation in order to avoid unwanted faults, and to document and remedy known ones

Analyze and execute the program in order to reproduce known failures

Demonstrate that the test object adheres and/or conforms to the necessary contractual, legal and regulatory requirements and standards

TERMINOLOGIES

Test Artifact: Any document produced during testing process such as test plans, test summary report.

Test Basis: Source of information (requirement) from which test cases are derived, defines the expected behavior of test object

Test Object: Set of test cases covering a single module

Test Item: A single test case in a test object

Test Condition: can be checked using one or more tests, can be a function, transaction or a structural element of a component

CONT.

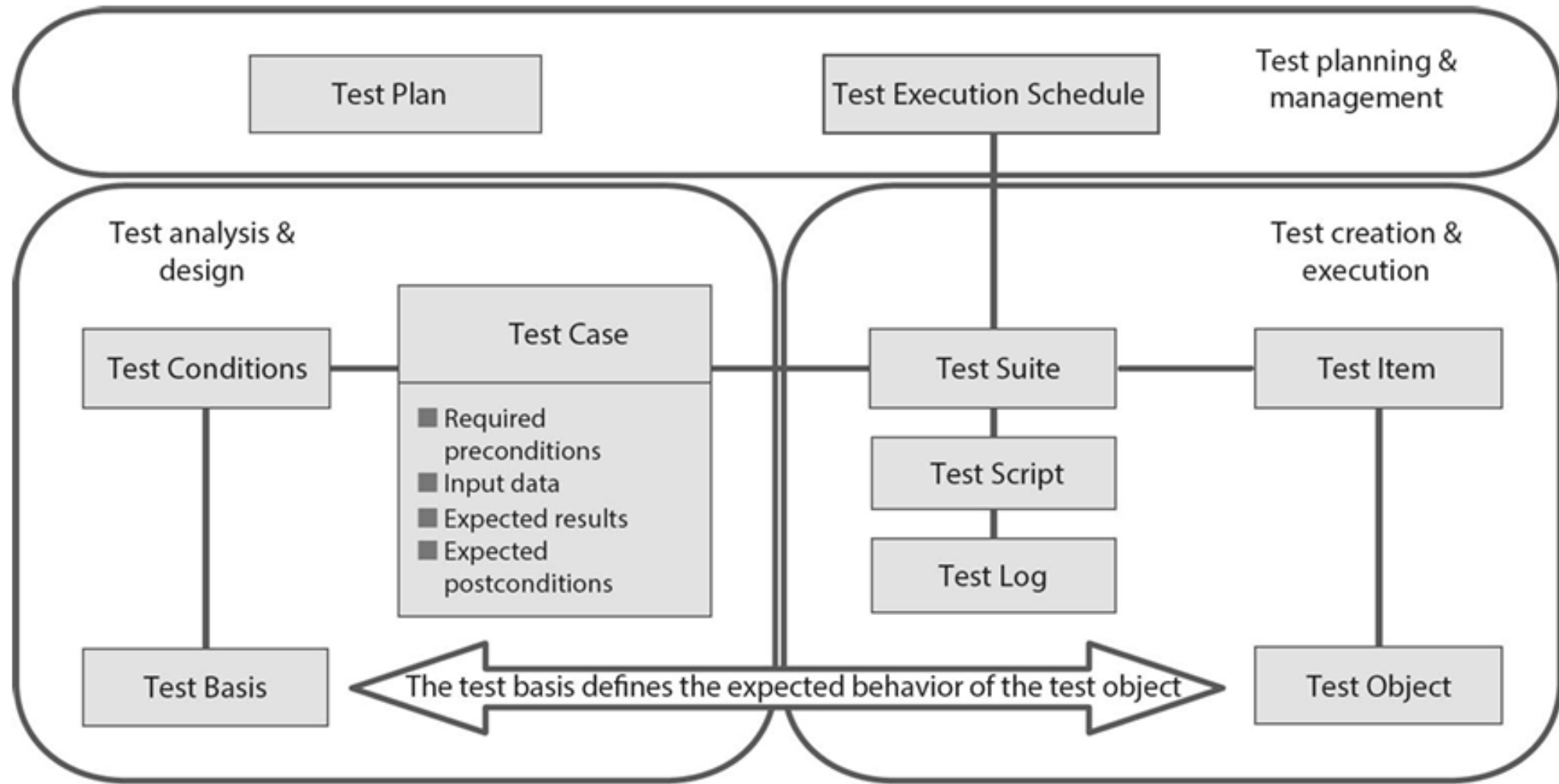
Test Run: Actual execution of a test case, test object is fed with test data and executed on computer

Test Suite: combined test cases that are executed in a test cycle

Test Execution Schedule: Timing of test cycles when test cases will be executed and after which test case

Test Logs: Test runs are logged and recorded in a test summary report

Test Plan: For every test object, a test plan is prepared which defines everything required to conduct the test



TESTING EFFORT

It is difficult to say just how much effort you should spend testing, as this depends very much on the nature of the project at hand

Can be specified as ratio of testers to developers (from one tester for every ten developers to three testers per developer)

Reasonable testing effort always depends on the degree of risk involved in failure and an evaluation of the danger this incurs

System parts with a high risk have to be tested more extensively

Risk assessment has to be carried out for the individual system parts or even for individual failure modes

CONT.

If there is a high risk of a system or subsystem malfunctioning, the test requires more effort than for less critical (sub) systems

A computer game that saves scores incorrectly can be costly for its manufacturer, as such faults affect the public acceptance of a game and its parent company's other products, and can lead to lost sales and damage to the company's reputation

CASE STUDY: RISKS AND LOSSES WHEN FAILURE OCCURS

The *DreamCar* module constantly updates and displays the price of the current configuration. Registered customers with validated ID can order a vehicle online.

Once a customer clicks the Order button and enters their PIN, the vehicle is ordered and the purchase committed. Once the statutory revocation deadline has passed, the chosen configuration is automatically passed on to the production management system that initiates the build process.

Because the online purchase process is binding, if the system calculates and displays an incorrect price the customer has the right to insist on the paying that price. This means that wrongly calculated prices could lead to the manufacturer selling thousands of cars at prices that are too low. Depending on the degree of miscalculation, this could lead to millions of dollars in losses. Having each purchase order checked manually is not an option, as the whole point of the VSR-II system is that vehicles can be ordered completely automatically online.

THE COST OF FAILURE

In March 2016, a concatenation of software faults destroyed the space telescope Hitomi, which was built at a cost of several hundred million dollars. The satellite's software wrongly assumed that it was rotating too slowly and tried to compensate using countermeasures. The signals from the redundant control systems were then wrongly interpreted and the speed of rotation increased continuously until the centrifugal force became too much and Hitomi disintegrated.

In 2018 and 2019 two Boeing 737 MAX 8 airplanes crashed due to design flaws in the airplane's MCAS flight control software. Here too, the software—misdirected by incorrect sensor information—generated fatal countermeasures.

APPLYING TESTING SKILLS EARLY ENSURES SUCCESS

At requirement stage, Identification and correction of flawed requirements reduces the risk of producing inappropriate or non-testable functionality

At design stage, increased awareness can reduce the risk of producing fundamental construction faults and makes it easier to design appropriate tests

At coding stage, better understanding of the code itself and the tests it requires reduces the risk of producing faulty code and of designing inappropriate tests

At testing stage, if testers verify and validate software before release, they are sure to identify and remedy additional faults that would otherwise remain undiscovered

BASIC PRINCIPLE OF TESTING

1. Testing shows the presence of defects, not their absence

- If an object doesn't fail during testing, this is no proof of freedom from faults or overall correctness.

2. Exhaustive testing is impossible

- It is impossible to design and perform a complete test suite that covers all possible combinations of input data and their preconditions.

3. Early testing saves time and money

- Helps to avoid (or at least reduce) the increasingly costly repair of faults later in the development lifecycle.

4. Defects cluster together

- Most defects can usually be found in a small number of modules. Testing effort can then be concentrated on the most relevant parts of the system.

CONT.

5. Beware the pesticide paradox

- Over time, tests become less effective the same way insects develop resistance to pesticides. Check your test cases regularly and, if necessary, modify them or add new ones.

6. Testing is context-dependent

- No two systems can be effectively tested the same way. An embedded system requires different tests than, for example, an e-commerce system. Testing in an agile project will be very different from that in a project based on a sequential life cycle model.

7. Absence-of-errors is a fallacy

- Even if you test all requirements comprehensively and correct all the faults you find, it is still possible to develop a system that is difficult to use. Prototyping and early involvement of a system's users are preventive measures used to avoid this problem.

QUALITY MANAGEMENT AND QUALITY ASSURANCE

Quality management (QM) covers all organizational activities and measures that serve to control quality

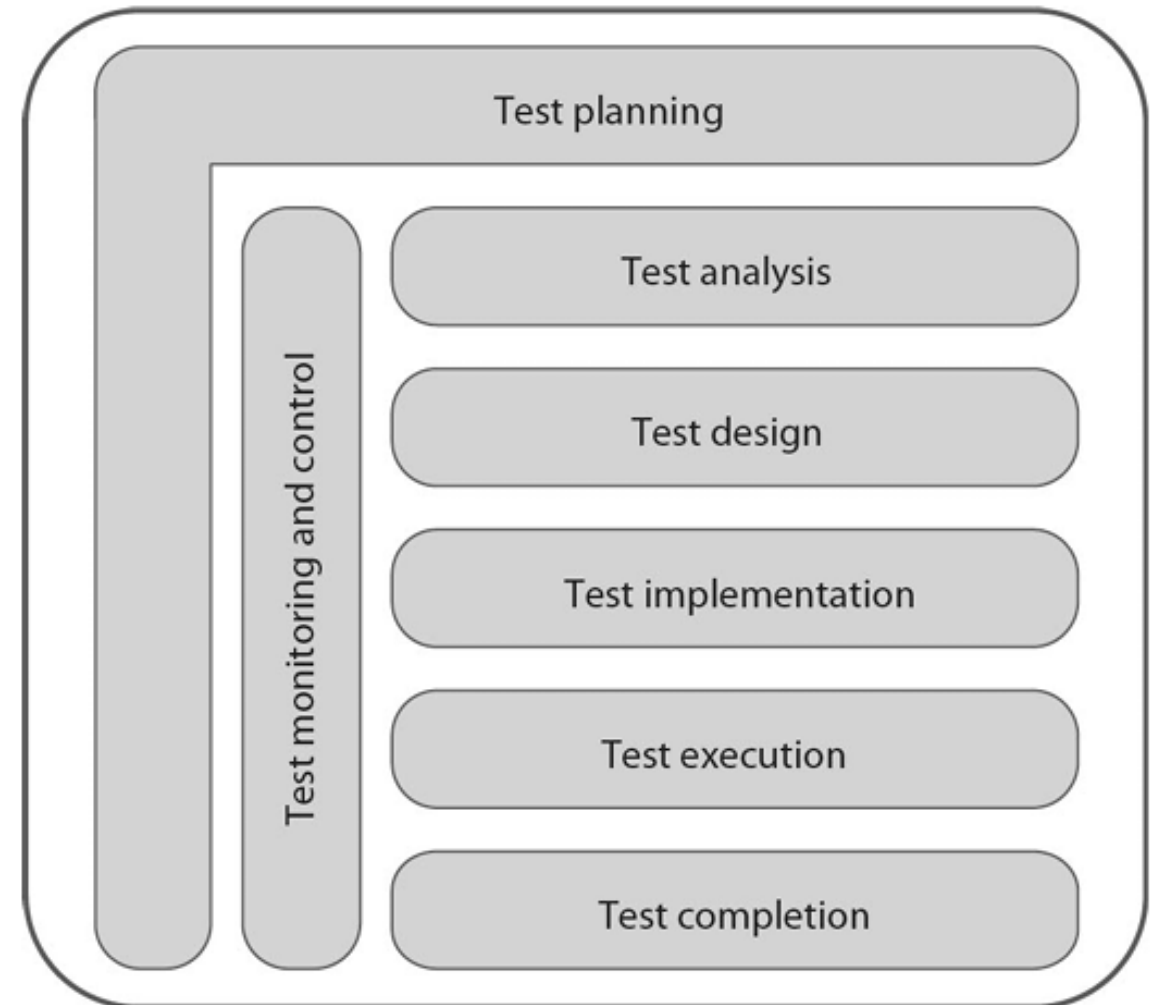
Quality assurance (QA) usually concentrates on measures which aim to ensure that specified procedures are applied and defined processes are adhered to

QA generates confidence in the fulfillment of quality requirements, which can be achieved by testing

QA is about making sure such processes are implemented and executed correctly

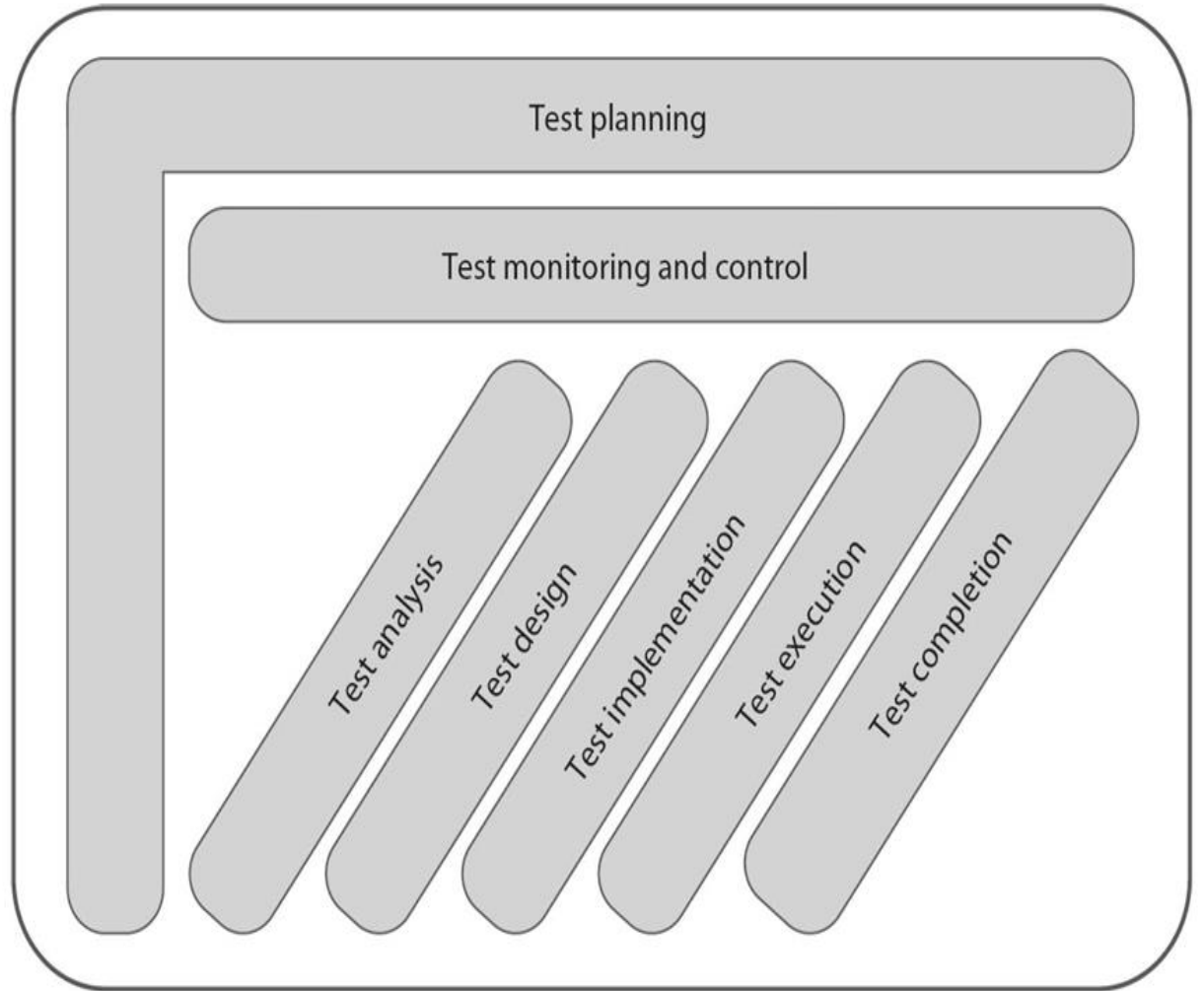
TESTING PROCESS

This model is quite unrealistic due to sequential activities



CONT.

This model accurately represents testing process due to time overlap between activities



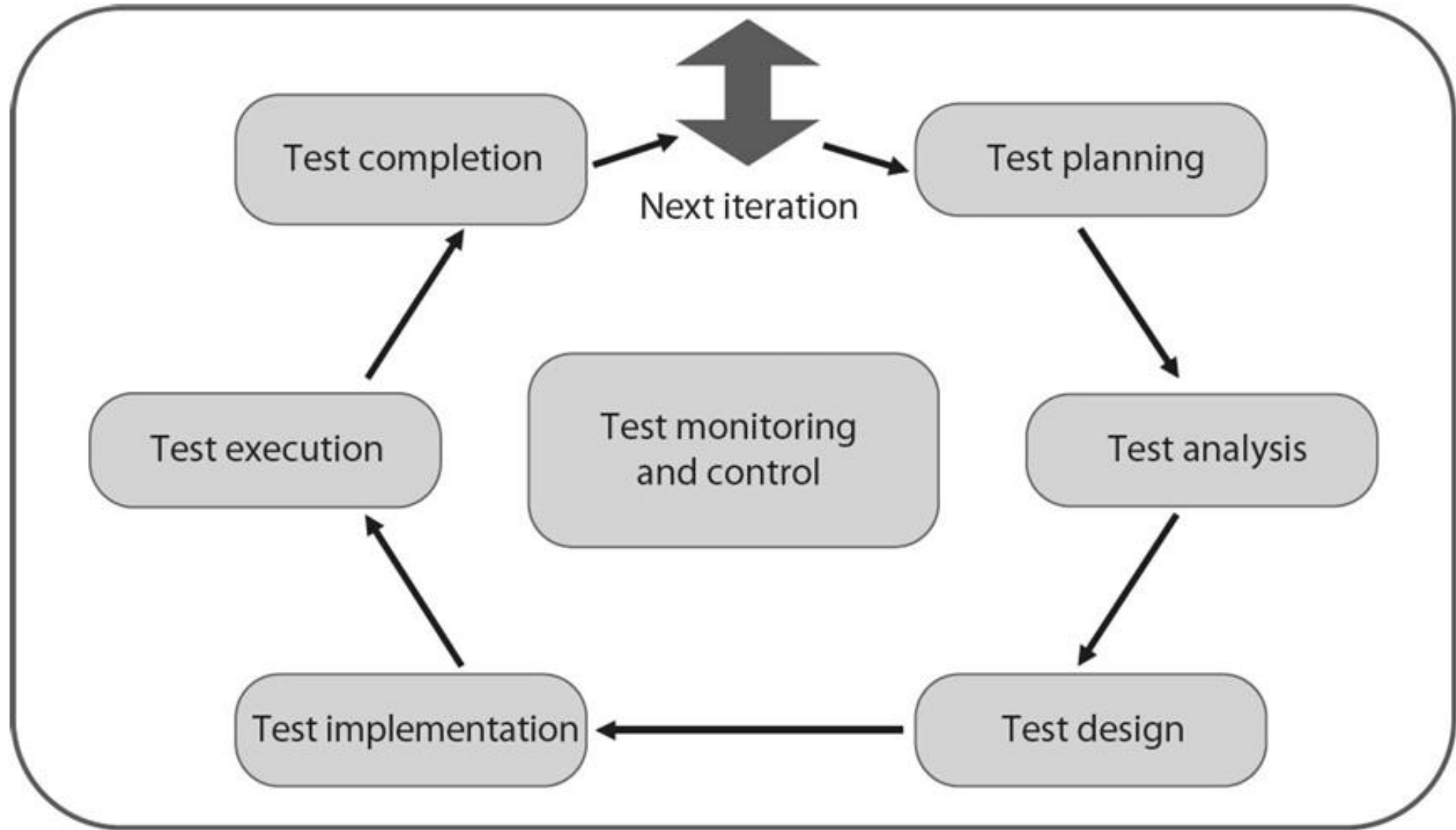
CONT.

A testing process is generally comprise of following activities:

- ☐ Test planning
- ☐ Test monitoring and control
- ☐ Test analysis
- ☐ Test design
- ☐ Test implementation
- ☐ Test execution
- ☐ Test completion

ITERATIVE TESTING

Software is often developed in small iterative steps—for example, agile development methods use continuous build and test iterations. The corresponding test activities therefore need to take place continuously and iteratively



TEST PLANNING

Testing will not work without a plan

Begins directly at the start of a software project

Update or adapt it to fit changing situations and project parameters

Adjusted throughout the entire product lifecycle

Created based on testing strategy chosen

Includes required resources, time to perform the corresponding activities

Also defines which tests are to be performed at which test level, makes sense to draft a separate test plan for each test level

CONT.

Coverage criteria: Verify when you have reached the testing objectives

Traceability: between test basis and result of test activities

Scheduling: Test schedule contains a list of all activities, tasks and/or events involved in the test process. It also includes planned start and end date for every activity and interdependencies between activities

TEST MONITORING AND CONTROL

Constant observation of the current testing activities compared with the planned activities

If the predefined criteria are fulfilled, the activity can be terminated

If risk evaluation is part of the test plan and you need to prove that you have sufficient risk coverage

If the required exit criteria have not been fulfilled by the tests you have performed, you need to design and execute additional tests

Any deviation from the original plan, these reports should also contain information regarding any prematurely terminated tests

TEST ANALYSIS

Involves determining what exactly needs to be tested

If a requirement doesn't specify the predicted result and/or system behavior precisely enough, then test cases cannot be simply derived

Cornerstone of the entire testing process is the test basis. If the test basis contains defects, you cannot formulate "correct" test conditions and you won't be able to draft "proper" test cases, whether it contains ambiguities, or whether there are gaps or omissions in the descriptions of functions

Once the test conditions have been identified and defined, you need to prioritize them, time restrictions often make it impossible to perform all the planned tests

Unambiguous bi directional traceability between the test basis and the results of your testing activities defines which test condition checks which requirement

Consider which test technique to use (i.e., black-box, white box, experience-based)

TEST DESIGN

Test conditions are used to create test cases (or sequences of test cases)

Test cases can be specified on two “levels”: abstract and concrete

An *abstract* (or “high-level”) test case doesn’t include specific input values or expected results, and is described using logical operators

Concrete (or “low-level”) test case uses specific input data and expected result values

Only concrete test cases can be executed on a computer

Abstract test cases can be derived from concrete cases too

Test oracle for predicting test results, specification can serve as oracle

EXAMPLE

A dealership can give its salespeople the option of discounts to apply to the price of a vehicle. For prices below \$15,000 there is no discount. For prices up to \$20,000, a discount of 5% is appropriate. If the price is below \$25,000, a 7% discount is possible. If the price is above \$25,000, a discount of 8.5% is to be applied. The above text enables us to derive the following relationships between price and discount:

$\text{Price} < 15,000$	$\text{Discount} = 0\%$
$15,000 \leq \text{Price} \leq 20,000$	$\text{Discount} = 5\%$
$20,000 < \text{Price} < 25,000$	$\text{Discount} = 7\%$
$\text{Price} \geq 25,000$	$\text{Discount} = 8.5\%$

CONT.

Abstract Test Cases	1	2	3	4
Input value x (price in \$)	$x < 15000$	$15000 \leq x \leq 20000$	$20000 < x < 25000$	$25000 \leq x$
Predicted result (discount in %)	0	5	7	8.5

CONT.

Concrete Test Cases	1	2	3	4
Input value x (price in \$)	14500	16500	24750	31800
Predicted result (discount in %)	$14500 \times 0/100 = 0$	$16500 \times 5/100 = 825$	$24750 \times 7/100 = 1732.5$	$31800 \times 8.5/100 = 2703$

CONT.

Test analysis has already prioritized the test conditions. These same priorities can be used and fine-tuned for your test cases (or sets of test cases)

Test infrastructure consists of all the organizational elements required for testing like test environment, testing tools, and appropriately equipped workstations

A test environment is required in order to run the test object on a computer

In order to avoid delays while testing, the test infrastructure should be up and running

TEST IMPLEMENTATION

Need to check that all additional tools (such as service virtualization, simulators, and other infrastructure elements) are in place and working

Test data have to be correctly transferred to the test environment

Planned tests have to be put into a logical sequence. This takes place according to the priorities you have defined

Test cases are grouped into test suites

Automating test procedures using scripts saves a lot of time compared with manual testing

TEST EXECUTION

Involves installing the test object in the test environment and checking that it can be started and run

Test execution process has to be precisely and completely logged. You need to log each test result

Verify that the overall testing strategy has been performed as planned

Logs should show which parts were tested, when, by whom, how intensively and with what results

Check that the corresponding failure has also been resolved and that no new failures have been caused by the correction process

Which requirements couldn't be verified due to failed tests or failures that were registered during testing

TESTING COMPLETION

Involves collating all the data collected by the completed test activities in order to evaluate the test experience and consolidate the testware and its associated materials

The test summary report aggregates all your testing activities and results, and includes an overall evaluation of the tests you have performed compared with your predefined exit criteria. The summary report is distributed to all stakeholders

TRACEABILITY

Between the test basis and test results

Checking whether all requirements have been covered by at least one test case

Traceability data helps you control the overall progress of the test process

Some companies build dedicated management systems that organize test results in a way that automatically delivers traceability data

THE INFLUENCE OF CONTEXT ON THE TEST PROCESS

Development life cycle model - Most agile models don't include detailed directions for testing and its associated activities

Test type - A test type is a group of test activities designed to test a component or a system for interrelated quality characteristics

Operational limitations - affect the structure and implementation of test processes including budget, deadlines, complexity, contractual and regulatory requirements

The context in which a software system is used also has an effect on the testing process. For example, software used exclusively for company-internal vacation planning will be subject to less thorough testing than a custom system built to control a client's industrial facility

Company guidelines and best practices can also have an effect on the testing process and the rules that govern it

EFFECTS OF HUMAN PSYCHOLOGY ON TESTING

Everyone makes mistakes, but nobody likes to admit it!

All failures found must be communicated to the developers

Discovering failures can be construed as criticism of a product or its author

Confirmation bias - tendency to search for, interpret, favor, and recall information in a way that confirms or strengthens your prior personal beliefs or hypotheses

Software developers generally assume that their code is fault-free, and confirmation bias makes it difficult for them to admit that their work might be flawed

Human trait is the tendency to “shoot the messenger” who brings bad news

A lot of developers are not aware that test results can help them hone their own skills, a way to reduce the risk of poor product quality

HOW TESTERS AND DEVELOPERS THINK

Developers design and produce a product, while testers verify and validate the product with a view to discovering faults and failures

Possible to improve product quality by combining these two different mindsets

Testers develop their soft skills through years of hands-on experience

Confirmation bias makes it more difficult for developers to recognize errors in their own work

Someone who both develops and tests a product requires a rare degree of critical distance to their own work

Developers should always know some testing basics, and testers should always have some development experience

CONCLUSION

Testing takes up a large proportion of a project's development resources. Precisely how much testing effort is required depends on the type of project at hand

The testing process—starting with planning and preparation steps— needs to be begun as early as possible in order to generate the maximum testing benefits within the project

It is important to recognize and observe the connections and the boundaries between testing, quality assurance, and quality management

The testing process comprises test planning, monitoring and control, analysis, design, implementation, execution, and completion. These activities can overlap and can be performed sequentially or in parallel. The overall testing process has to be adapted to fit the project at hand

CONCLUSION

Bidirectional traceability between the results of the individual testing activities ensures that you can make meaningful statements about the results of the testing process, and to make a reasonable estimate of how much effort will be involved in making changes to the system. Traceability is also critical to effective test monitoring and control.

All of the many factors that influence testing within an organization have to be considered. People make mistakes but don't usually like to admit it! This is why psychological issues play a significant role in the overall testing process.

The mindsets of testers and developers are very different, but both can benefit by learning from one another